Automatic Bug Detection and Classification in Full-Stack Applications Using Machine Learning

Kapil Kumar Gupta
Computer Science & Engineering Department
Amity University, Lucknow, Uttar Pradesh, India
kkgupta@lko.amity.edu

Arpita
Computer Science & Engineering Department
Amity University, Lucknow, Uttar Pradesh, India
arpita9601@gmail.com

***Abstract*— As full-stack applications become more complex, the need for efficient software defect identification, classification and prioritization becomes more urgent. Prior studies have investigated machine learning-based bug severity prediction, static source-code analysis, and automated error-log analysis. The practical challenge, however, is how to combine these complementary approaches into accessible and modular workflows. Furthermore, the class imbalance of bug-report datasets may cause misleading performance evaluation when the classification accuracy is reported without class-wise metrics. In this paper, we present BugSense AI, a hybrid framework for software bug analysis that combines machine learning-based severity prediction, static source-code inspection and rule-based error-log classification within a single web application. We propose a framework that employs Term Frequency-Inverse Document Frequency (TF-IDF) features and a Linear Support Vector Machine (SVM) to classify textual bug reports into six severity categories. For Python code, Abstract Syntax Tree (AST) analysis and Ruff are used, whereas for JavaScript code, ESLint is used. Severity of identified supported error patterns in application logs is estimated by a rule-based module. The framework is implemented as an interactive web-interface using Flask. We tested the approach on a cleaned test set of 17,317 bug reports and obtained an accuracy of 77.50%, a weighted F1-score of 0.7763 and a macro F1-score of 0.3826. Additionally, 8 analyzer checks and 10 API tests were passed, showing functional correctness for the tested scenarios. The findings emphasize the feasibility of integrating complementary bug-analysis methods and the limitations of severity classification on imbalanced datasets. The framework provides a practical basis for integrated software defect analysis and at the same time uncovers opportunities for improved classification and diagnostic capabilities.**

***Keywords* — Software Bug Detection, Bug Severity Classification, Machine Learning, Support Vector Machine, TF-IDF, Static Code Analysis, Error Log Analysis, Software Quality Assurance.**

Introduction

The growing complexity of full-stack applications has made the detection and classification of software defects an important aspect of software quality assurance. Modern applications consist of frontend interfaces, backend services, databases and APIs. Programming errors, unexpected runtime behaviors or component failures can cause defects. Debugging usually involves manual review of bug reports, source code and application logs, which is a time consuming process to identify and prioritize defects.

Automating the classification of software bugs has been widely researched using machine learning (ML). Previous studies [1], [2] have shown the effectiveness of text mining and supervised learning techniques in predicting bug-report severity. However, the severity classification is a challenging task because of the highly imbalanced nature of software defect datasets that may cause misleading accuracy scores and poor predictions of the underrepresented classes. Static code analysis and automated log inspection are complementary techniques to detect potential programming defects and runtime errors.

Some of these tasks are addressed in existing research and software-quality platforms. The challenge of practical engineering is to combine complementary analysis techniques in a lightweight and modular workflow. Also, the severity prediction evaluation must focus on class-wise performance, not just overall accuracy. In this study, we investigate these issues using BugSense AI, a hybrid software bug-analysis framework that combines machine-learning-based severity prediction, static source-code inspection, and rule-based error-log classification.

In the proposed system, the feature extraction of textual bug reports is done by the Term Frequency-Inverse Document Frequency (TF-IDF) and Linear Support Vector Machine (SVM) classifier is used for categorizing textual bug reports into six severity levels. We analyze the source code of Python and JavaScript using Python Abstract Syntax Tree (AST), Ruff and ESLint and classify application logs based on pre-defined patterns and heuristic rules. All components are integrated via a Flask REST API and an interactive web interface.

The main contributions of this work are:

a unifying architecture that supports three complementary bug-analysis workflows; severity classification pipeline based on SVM evaluation using accuracy and class-sensitive metrics; and functional validation of source-code analysis, error-log categorization, and API integration. The framework is evaluated on a cleaned test set of 17,317 bug reports with 77.50% accuracy, 0.7763 weighted F1-score and 0.3826 macro F1-score. The results show the limitations of severity prediction in the presence of class imbalance.

I. Literature Review

Software defect analysis has been proposed using machine-learning based classification of bug reports, static analysis

of the source code, and analysis of the application logs. In [1], the authors proposed a method of using text-based classification of bug reports to predict the severity of the reported bug. In a subsequent work [2], they investigated the performance of different data-mining algorithms in this classification task. Another approach to bug-report

prioritization was proposed by Tian et al. [3], who considered additional factors beyond the text descriptions of the bugs. Recent advances in natural language processing (NLP) have allowed for the proposal of contextual representations of text related to software. BERT [4], proposed by Devlin et al., is a language representation model based on the transformer architecture. CodeBERT [5] is a similar model developed for programming languages and natural language. These methods offer alternatives to the standard TF-IDF representations of text, but require additional computational cost.Static analysis tools for source code, such as Ruff for Python and ESLint for JavaScript [6], [7], perform automated checks of the source code for potential errors. Additionally, Du et al. [8] proposed a system called DeepLog that can automatically detect anomalies in system logs. These methods target different aspects of software quality, and not the same classification tasks.While integrated bug-analysis systems have been proposed in the past, there are opportunities to explore different architectures for such systems. BugSense AI incorporates three different analysis methods, based on TF-IDF with SVM classification of bug-report severity, static analysis of source code, and rule-based classification of log files. Beyond the classification accuracy of these systems, the proposed system in this work also reports the F1-scores (both macro and weighted) to analyze the effect of class imbalance.

Study	Approach	Relevance to This Work
Lamkanfi et al. [1], [2]	Text mining and severity classification	Foundation for ML-based severity prediction
Tian et al. [3]	Multi-factor priority prediction	Highlights additional bug-report features and class imbalance
Devlin et al. [4]	BERT language representations	Contextual NLP alternative
Feng et al. [5]	CodeBERT representations	Potential future code-ML extension
Du et al. [8]	Deep-learning-based log anomaly detection	Demonstrates learning-based log analysis

Table 1: Summary of related work

II. METHODOLOGY

In this paper, we propose BugSense AI which is a hybrid framework that combines the static source-code analysis, machine-learning based bug severity prediction and rule-based error-log classification. The system is built using a modular architecture where user inputs are processed by the appropriate analysis module through a REST API developed using Flask. The whole process is shown in Fig. 1.

A. Design of the System

The framework has three modules for analysis. The first one uses a trained Support Vector Machine (SVM) classifier to predict the severity of textual bug reports. The second does static analysis of Python and JavaScript source code, using language-specific tools. The third identifies predefined patterns of errors in the application logs and estimates their severity. All the tools have a web interface to let the user submit inputs and see the relevant findings like severity, category, priority and detected issues if any.

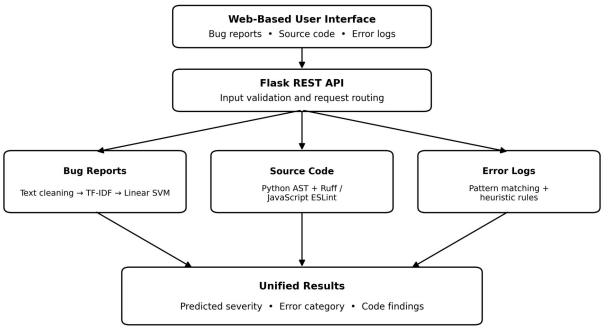


Fig. 1. Proposed BugSense AI Architecture

B. Dataset Preparation and Text Preprocessing

The severity classification module is applied on labeled software bug reports with their textual description and associated severity categories. The dataset was split into training and testing sets: 70362 bug reports were used for training the severity classification module and 17591 reports were reserved for the testing set. Text preprocessing was applied to the bug reports. Bug report text was lowercased, and URLs and special characters were removed from the text. Additionally, whitespace within the text was normalized. To eliminate bug reports that are very similar to the bug reports in the testing set, 274 near-duplicate bug reports (identified with a character-level TF-IDF similarity threshold of 0.90) were removed from the testing set. The resulting testing set contains 17317 bug reports.

C. Classification of Severity Using Machine Learning

We converted the preprocessed bug descriptions into numerical feature representations using word- and character-level TF-IDF features. The generated vectors were fed into a Linear Support Vector Machine classifier. The deployed model uses balanced class weights, regularization parameter $C=0.5$ and a maximum of 5000 training iterations.

The TF-IDF weight of a term t in document d can be expressed as

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

where TF is the term frequency and IDF is the inverse frequency of the document. The classifier classifies each bug report into one out of six severity categories: blocker, critical, major, normal, minor, or trivial.

D. Static Source Code Analysis

The source code analysis module supports Python and JavaScript. Python code is inspected using Abstract Syntax Tree (AST) processing and Ruff, and JavaScript code is inspected using ESLint. These tools detect supported defect patterns and potentially unsafe constructs such as division

by zero, dynamic code execution and non-strict equality comparisons.

For each problem found the module outputs available diagnostic information including the source line, rule ID, description and assigned severity. The overall severity is estimated using the detected findings . This module uses static analysis, not machine-learning based code classification.

E. Error Log Classification Based on Rules

The error-log analysis module uses predefined patterns and heuristic rules to analyze textual application logs. It identifies supported error categories, like database and authentication failures, and assigns an estimated severity and priority. This component differs from the bug-report classifier in that it does not use a trained machine-learning model. Any input that is not recognized can be labeled as unknown.

F. Implementation and Assessment

The system was implemented with Python and Flask for the backend processing, Scikit-learn for machine-learning classification, and HTML, CSS, and JavaScript for the front end. The /api/analyze endpoint routes submitted inputs to the correct analysis module and returns structured JSON responses.

For the severity classifier, we also used accuracy, precision, recall and F1-score as well as macro- and weighted-average metrics to address the class imbalance. Functional validation included eight analyzer checks and ten API tests, and browser-based demonstrations of the three analysis modes.

III. RESULT AND DISCUSSION

The performance of BugSense AI was assessed through machine-learning classification metrics, functional analyzer tests and API validation. The evaluation was on the effectiveness of the prediction of bug severity and the functional behavior of the integrated source-code and error-log analysis modules.

A. Performance of Bug Severity Classification

The Advanced SVM classifier's classification capabilities are tested on an original test set of 17,591 bug reports and a cleaned test set of 17,317 reports after the removal of 274 near duplicate reports. The performance of the final model can be found in Table II.

Evaluation Metric	Original Test Set	Cleaned Test Set
Accuracy (%)	77.72	77.50
Macro Precision	—	0.3814
Macro Recall	—	0.3850
Macro F1-score	0.4040	0.3826
Weighted F1-score	0.7785	0.7763
Test Samples	17,591	17,317

Table II. Performance of the advanced SVM classifier

The overall classification performance of the classifier obtained 77.50% accuracy on the cleaned test set. But the macro F1-score is much lower, showing inconsistent performance across the severity classes. This disparity is mainly caused by the imbalanced distribution of classes, since the normal category accounts for about 81.6% of the reports.

B. Performance Per Class

To study the effect of class imbalance, F1-score was computed for each severity category separately as shown in Table III .

Severity Class	F1-score
Blocker	0.2588
Critical	0.6588
Major	0.1749
Minor	0.1318
Normal	0.8734
Trivial	0.1978

Table III. Class-wise F1-score

The normal class achieved the highest F1-score of 0.8734 followed by the critical class of 0.6588. The contrast scores for the major, minor and trivial classes were significantly lower. The results indicated that the use of balanced class weighting was insufficient to address the challenges associated with predicting the minority class. Therefore, the overall accuracy of the classifier should not be taken as an indicator of the accuracy with which severity can be classified.

C. Functional Testing of Integrated Framework

The source-code and error-log analysis modules were tested using a variety of functional tests. The python analysis identified supported patterns such as division by zero and dynamic code execution. The java script analysis identified rule violations such as var, non-strict equality and eval(). The error-log module was able to correctly classify a database connection timeout error as a database error with a severity of major.

Test Component	Result
Analyzer assertion tests	8/8 passed
Flask API tests	10/10 passed
Browser workflow examples	4/4 demonstrated successfully

Table IV. Results of functional validation

The API tests included requests with valid inputs, requests to test the health of the API, requests with unsupported inputs, requests with empty inputs and requests with inputs that exceed the length limits of the API. The browser demonstrations showed that the interface displayed the severity of the inputs, the description of the issues with the inputs, the identifiers of the rules that were violated by the inputs and the categories of the errors that were detected in the inputs. These tests proved that the implemented API functions as expected for the tested inputs but did not prove that the implemented API can correctly detect all possible errors in the inputs.

D. System Outcomes and Functional Results

Input Type	Test Scenario	System Outcome	Severity
Bug Report	Application crashes during login	ML-based severity prediction	Major
Python Code	Division by zero and eval() usage	2 issues detected	Critical

JavaScript Code	var, ==, and eval() usage	3 issues detected	Critical
Error Log	MySQL database connection timeout	Database Error; High priority	Major

Table V. Observed outputs of the Bugsense AI Framework

To evaluate the functional performance of BugSense AI, representative inputs were applied to its three analysis modules. As Table V shows, the machine learning classifier identified a login-related application crash as Major. Python static analyzer found two possible problems related to division by zero and dynamic code execution. The static analyzer of JavaScript found three potential problems concerning variable declaration, use of the equality operator and dynamic code execution. Finally, the rule-based log analyzer detected a Database Error, a database connection timeout, with a severity level of Major and a priority level of High. The results demonstrate that the BugSense AI framework is capable of analyzing various types of software defects and producing a consolidated analysis report for each of them.

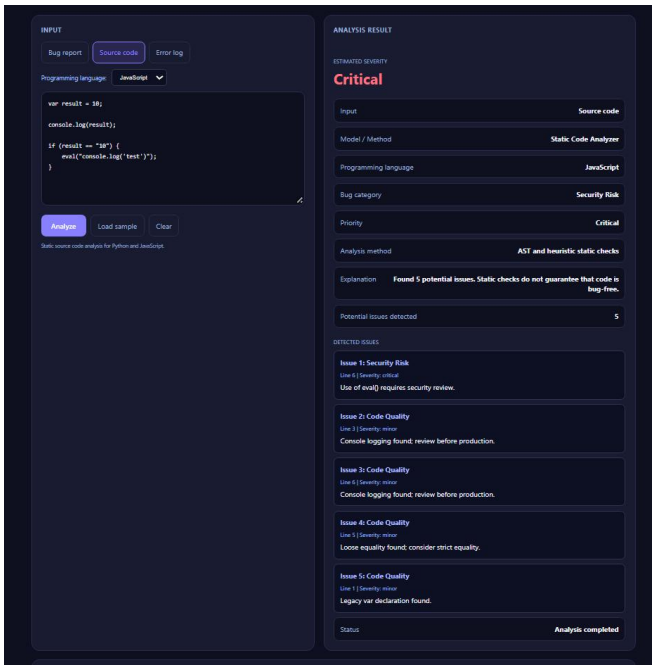


Fig. 2. BugSense AI dashboard shows problems found in Python code and how serious each issue is.

E. Discussion and Limitations

The experiments show that it is feasible to combine machine-learning based severity classification, static source-code inspection and rule-based error-log categorization into a single web application. The modular architecture provides a common interface through which different types of software information can be processed while retaining the methods of analysis appropriate to each type of input.

There are, though, some limitations. The severity classifier performs worse on underrepresented classes, and static analysis is limited to supported languages and detection rules as configured. Similarly, rule-based log classification relies on predefined error patterns and could miss unknown failures. Also, the current evaluation does not show any improvement in debugging time, developer productivity or performance over the existing integrated platforms. These limitations indicate the need for larger datasets, additional validation and more comprehensive comparative evaluation.

IV. Conclusion And Future Works

We proposed BugSense AI, a framework for automated software bug analysis using a combination of machine learning-based severity prediction, static source code inspection and rule-based error log classification techniques. The system classifies bug reports into six severity categories using machine learning algorithms with TF-IDF and Linear SVM algorithms, and source code inspection using Python AST, Ruff and ESLint. The system was implemented as a web application using Flask.

The experimental evaluation of BugSense AI on the 17317 test reports obtained an accuracy of 77.50%, weighted F1-score of 0.7763 and macro F1-score of 0.3826. The results show the problems of class imbalance in the test reports. For the BugSense AI framework, there were eight successful analyzer tests and 10 successful API tests.

Future work for BugSense The AI system will tackle the class imbalance issue by applying techniques such as enhanced feature engineering or resampling methods to enhance the classification performance of the minority class. Further possible extensions for the framework include the use of pretrained code models such as CodeBERT to perform machine learning based source code defect classification, the use of supervised learning techniques for intelligent error log classification, and the integration of evidence based root cause analysis techniques for correlating bug reports, source code and application logs. These potential extensions will require additional datasets for training and evaluation.

To summarize, the proposed BugSense AI framework for automated software defect management shows the feasibility and can be used as a baseline for further research in the area.

V. References

[1] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, “Predicting the severity of a reported bug,” in Proc. 7th IEEE Working Conf. Mining Software Repositories (MSR), 2010, pp. 1–10, doi: 10.1109/MSR.2010.5463284.

[2] A. Lamkanfi, S. Demeyer, Q. D. Soetens, and T. Verdonck, “Comparing mining algorithms for predicting the severity of a reported bug,” in Proc. 15th European Conf. Software Maintenance and Reengineering (CSMR), 2011, pp. 249–258, doi: 10.1109/CSMR.2011.31.

[3] Y. Tian, D. Lo, X. Xia, and C. Sun, “Automated prediction of bug report priority using multi-factor analysis,” Empirical Software Engineering, vol. 20, no. 5, pp. 1354–1383, 2015, doi: 10.1007/s10664-014-9331-y.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for

language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186, doi: 10.18653/v1/N19-1423.

[5] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 1536–1547, doi: 10.18653/v1/2020.findings-emnlp.139.

[6] Astral, "Ruff documentation: Rules," [Online]. Available: https://docs.astral.sh/ruff/rules/. Accessed: Oct. 9, 2026.

[7] ESLint, "ESLint rules reference," [Online]. Available: https://eslint.org/docs/latest/rules/. Accessed: Oct. 9, 2026.

[8] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2017, pp. 1285–1298.